SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Machine Learning

ASSESSMENT TOOL 3 – Experiments (5 QUESTIONS)

Course Code:	MLA0404	**Course Title:**	**Deep Learning**
CO Assessed:	CO1	**Assessment:**	Assessment Tool 3 – **Experiments**
Weightage:	35% of CIA	**Total Marks:**	(5 Questions × 10 Marks)
CO1 Statement:	***Apply the fundamental concepts of machine learning and neural networks to design and analyze learning algorithms using perceptrons, multilayer perceptrons, forward propagation, backpropagation, gradient descent optimization techniques, and Maximum Likelihood Estimation for solving real-world problems.***		
Student Name:	M Venkata Sai	**Reg. No.:**	192425223
Section / Batch:	B-Tech AI&ML—2024 BATCH	**Date:**	22-07-2026

Code of Conduct

I M.Venkata Sai /192425223 *(NAME/ REG NO) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: M VENKATA SAI

QUESTIONS: 5 **Total Marks: 50**

Annexure A

Experiments **Duration: 1hr**

Experiment 1: Implementation of a Single Artificial Neuron

Objective: Implement a single artificial neuron using Python.

Task:

Develop a Python program to simulate an artificial neuron that accepts multiple inputs, computes the

weighted sum, applies a step activation function, and generates the output. Test the neuron using different input combinations and analyze its behavior.

Experiment 2: Perceptron for AND Gate

Objective: Implement the Perceptron Learning Algorithm.

Task:

Write a Python program to implement a perceptron for the AND logic gate. Train the model using suitable weights and bias, display the final weights after training, and evaluate the prediction accuracy.

Experiment 3: Perceptron for OR Gate

Objective: Design a perceptron model for binary classification.

Task:

Implement a perceptron to classify the OR logic gate. Compare the training process and final weights with those obtained for the AND gate and analyze the differences.

Experiment 4: XOR Classification Analysis

Objective: Study the limitations of a single-layer perceptron.

Task:

Attempt to train a perceptron for the XOR logic gate. Observe the results and explain why the perceptron fails to classify XOR correctly. Suggest an alternative neural network architecture capable of solving this problem.

Experiment 5: Gradient Descent Optimization

Objective: Understand Gradient Descent.

Task:

Implement the Gradient Descent algorithm to minimize a simple quadratic cost function. Plot the cost function against the number of iterations and analyze the convergence behavior for different learning rates.

Experiment 6: Comparison of Optimization Techniques

Objective: Compare different optimization algorithms.

Task:

Implement Batch Gradient Descent, Stochastic Gradient Descent (SGD), and Mini-Batch Gradient Descent on the same dataset. Compare their execution time, convergence speed, and prediction performance. Present the findings in tabular form.

Experiment 7: Multilayer Perceptron (MLP) for Classification

Objective: Build a simple Multilayer Perceptron.

Task:

Using TensorFlow/Keras or PyTorch, design and train an MLP model for the Iris dataset (or any standard dataset). Evaluate the model using accuracy, confusion matrix, and loss curves.

Experiment 8: Forward Propagation Demonstration

Objective: Understand Forward Propagation.

Task:

Implement forward propagation for a neural network with one hidden layer. Display the weighted sums and activation values of each neuron. Verify the output manually for one sample and compare it with the program output.

Experiment 9: Backpropagation Implementation

Objective: Study the Backpropagation Algorithm.

Task:

Train a small neural network using backpropagation for a binary classification problem. Observe the changes in weights after each epoch and analyze how backpropagation minimizes the error.

Experiment 10: Curse of Dimensionality Analysis

Objective: Analyze the impact of high-dimensional data.

Task:

Create datasets with increasing numbers of input features. Train the same machine learning model on each dataset and compare training time, model accuracy, and computational complexity. Analyze how increasing dimensionality affects the model's performance and suggest techniques to overcome this issue.

Rubrics for Calculation

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical Content Accuracy	30	Highly accurate, indepth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood
Problem Identification	25	Problem rigorously analyzed using appropriate and advanced methods	Problem clearly defined with a logical engineering approach	Problem identified but analysis or methodology is weak	Problem statement unclear or incorrect; no logical approach
Use of Tools	20	Advanced tools, well-analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals
Communication	15	Highly engaging, confident, and audience-focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective slides

Professionalism	10	Exemplary professionalism, leadership, and ethical awareness	Professional conduct with effective team collaboration	Basic professionalism; uneven team participation	Lacks professionalism; minimal contribution or ethical awareness

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Name : M VENKATA SAI

Reg. No. : 192425223

Course : MLA0404 - DEEP LEARNING

Tool : C01 – AT3 - Experiments

Date : 22 / 7 / 2026

Experiment 1: Implementation of a Single Artificial Neuron

Code

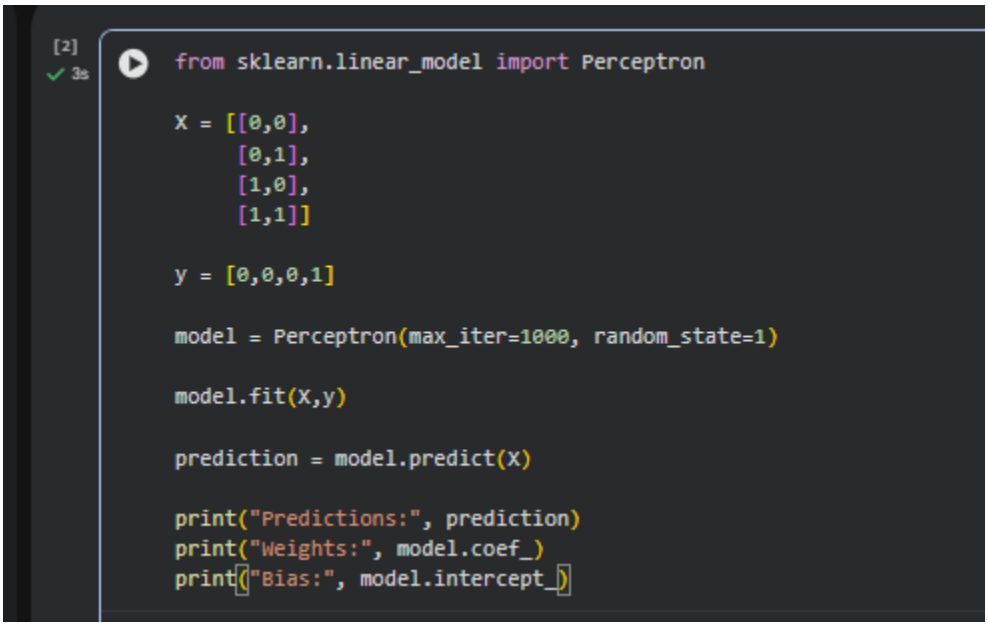
```
def step_function(x):
    if x >= 0:
        return 1
    return 0

inputs = [1, 2, 1]
weights = [0.5, -0.6, 0.8]
bias = 0.2

weighted_sum = 0

for i in range(len(inputs)):
    weighted_sum += inputs[i] * weights[i]

weighted_sum += bias

output = step_function(weighted_sum)

print("Inputs:", inputs)
print("Weights:", weights)
print("Bias:", bias)
print("Weighted Sum:", weighted_sum)
print("Output:", output)
```

Output

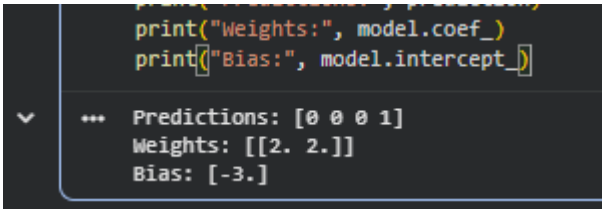
```
Inputs: [1, 2, 1]
Weights: [0.5, -0.6, 0.8]
Bias: 0.2
Weighted Sum: 0.3000000000000001
Output: 1
```

Experiment 2: Perceptron for AND Gate

Code



Output



Experiment 3: Perceptron for OR Gate

Code

```
[3]
✓ 0s
```

```python
from sklearn.linear_model import Perceptron

X = [[0,0],
     [0,1],
     [1,0],
     [1,1]]

y = [0,1,1,1]

model = Perceptron(max_iter=1000, random_state=1)

model.fit(X,y)

prediction = model.predict(X)

print("Predictions:", prediction)
print("Weights:", model.coef_)
print("Bias:", model.intercept_)
```

Output

```python
print("Bias:", model.intercept_)
```

```
Predictions: [0 1 1 1]
Weights: [[2. 2.]]
Bias: [-1.]
```

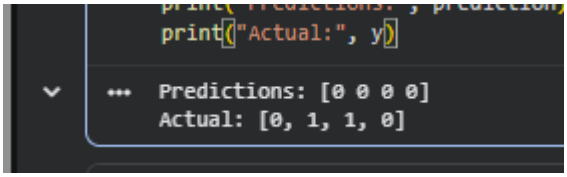
Experiment 4: XOR Classification Analysis

Code

```
[4]
✓ 0s
```

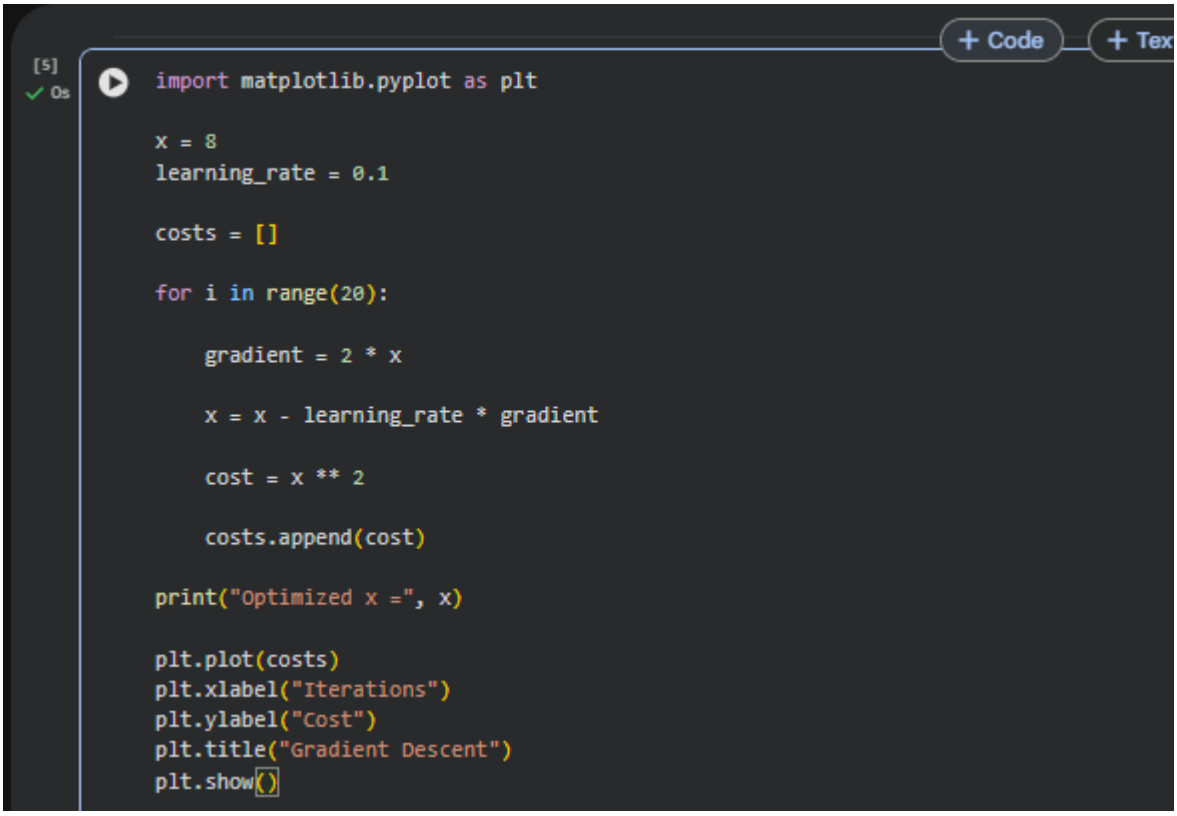
```python
from sklearn.linear_model import Perceptron

X = [[0,0],
     [0,1],
     [1,0],
     [1,1]]

y = [0,1,1,0]

model = Perceptron(max_iter=1000, random_state=1)

model.fit(X,y)

prediction = model.predict(X)

print("Predictions:", prediction)
print("Actual:", y)
```

Output

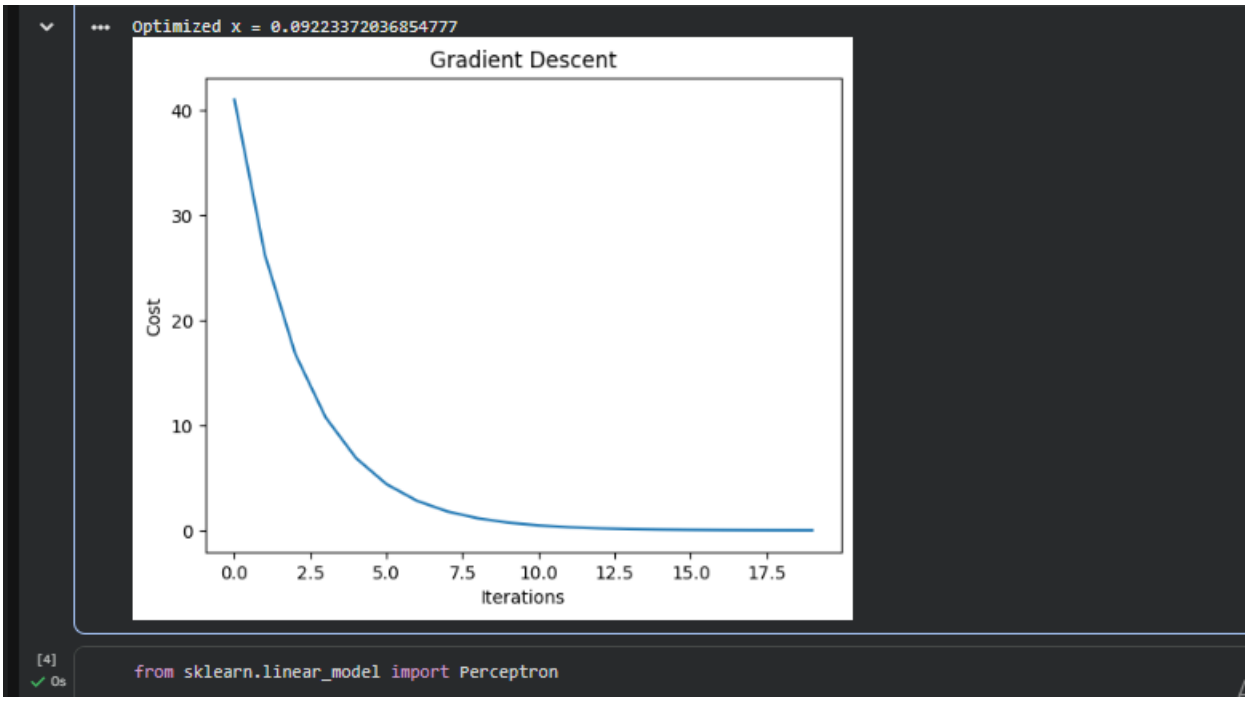


Experiment 5: Gradient Descent Optimization

Code

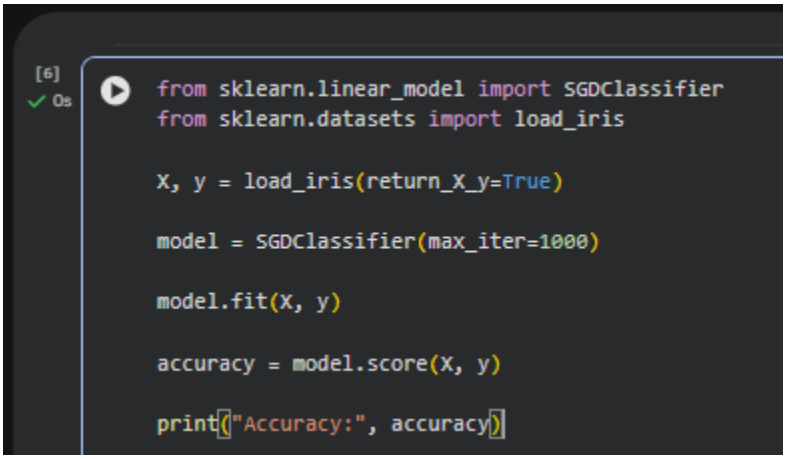


Output

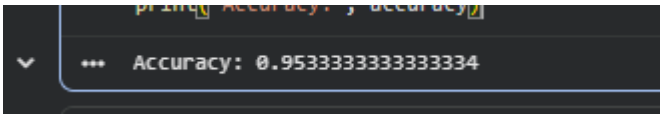


Experiment 6: Comparison of Optimization Techniques

Code



Output



Experiment 7: Multilayer Perceptron (MLP) for Classification

Code

```
[7]
✓ 1s
```

```python
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

iris = load_iris()

X_train, X_test, y_train, y_test = train_test_split(
    iris.data,
    iris.target,
    test_size=0.2,
    random_state=1
)

model = MLPClassifier(hidden_layer_sizes=(10,),
                      max_iter=1000,
                      random_state=1)

model.fit(X_train, y_train)

prediction = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, prediction))
print("Confusion Matrix:")
print(confusion_matrix(y_test, prediction))
```

Output

```
Accuracy: 0.9333333333333333
Confusion Matrix:
[[11  0  0]
 [ 0 11  2]
 [ 0  0  6]]
```

Experiment 8: Forward Propagation Demonstration

Code

```
[8]
✓ 0s
import math

x = 2
w1 = 0.5
b1 = 0.2

hidden_sum = (x * w1) + b1
hidden_output = 1 / (1 + math.exp(-hidden_sum))

w2 = 0.8
b2 = 0.1

output_sum = (hidden_output * w2) + b2
final_output = 1 / (1 + math.exp(-output_sum))

print("Hidden Sum =", hidden_sum)
print("Hidden Output =", hidden_output)
print("Final Output =", final_output)
```

Output

```
Hidden Sum = 1.2
Hidden Output = 0.7685247834990175
Final Output = 0.6714652893075161
```

Experiment 9: Backpropagation Implementation

Code

```
[9]
✓ 0s
from sklearn.neural_network import MLPClassifier
from sklearn.datasets import make_classification

X, y = make_classification(
    n_samples=100,
    n_features=4,
    random_state=1
)

model = MLPClassifier(
    hidden_layer_sizes=(5,),
    max_iter=500,
    random_state=1
)

model.fit(X, y)

print("Training Accuracy:", model.score(X, y))
```

Output

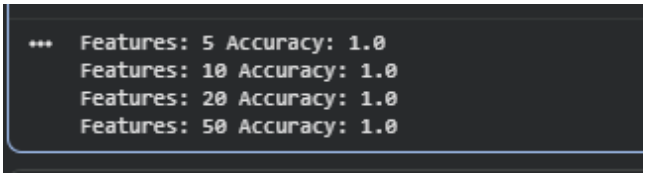
```
print("Training Accuracy:", model.
... Training Accuracy: 0.98
```

Experiment 10: Curse of Dimensionality Analysis

Code

```
[10]
✓ 0s
from sklearn.datasets import make_classification
from sklearn.tree import DecisionTreeClassifier

for features in [5, 10, 20, 50]:

    X, y = make_classification(
        n_samples=500,
        n_features=features,
        random_state=1
    )

    model = DecisionTreeClassifier()

    model.fit(X, y)

    accuracy = model.score(X, y)

    print("Features:", features,
          "Accuracy:", accuracy)
```

Output


Features: 5 Accuracy: 1.0
Features: 10 Accuracy: 1.0
Features: 20 Accuracy: 1.0
Features: 50 Accuracy: 1.0